DuocAIAgent

Agente de IA para la orientación académica y estudiantil en Duoc UC

Profesor: Francisco Andres Macaya

Integrantes

- Joaquin Herrera
- Roberto Bustos

Planteamiento del problema

El ingreso a la educación superior supone enfrentar un volumen considerable de información normativa y administrativa distribuida en distintas fuentes. En Duoc UC, el reglamento académico, el calendario académico y la resolución de una consulta habitual exige cruzar información de varios documentos de la institución. Esta dispersión dificulta que los estudiantes nuevos encuentren respuestas oportunas y confiables.

Como consecuencia, durante los meses de marzo y julio, el punto estudiantil recibe consultas reiteradas sobre toma de ramos, prácticas y reprobación de asignaturas. El requerimiento organizacional consiste en automatizar este flujo de atención: responder con fuentes oficiales, elaborar un plan con fechas concretas para cada estudiante y derivar a coordinación únicamente los casos que no pueden resolverse de forma automática.

Desarrollo de un sistema RAG

Como primera solución se desarrolló un sistema RAG (*Retrieval-Augmented Generation*) en la Evaluación Parcial N°1. Este enfoque combina recuperación semántica con un modelo de lenguaje: los documentos institucionales se representan como vectores (*embeddings*), ante cada pregunta se recuperan los fragmentos más similares y estos se entregan como contexto al modelo, que redacta una respuesta fundamentada en fuentes oficiales.

El RAG operaba bajo un esquema lineal de un solo paso (pregunta → búsqueda → respuesta), con un historial limitado a 6 mensajes y capacidades exclusivamente de lectura. Si bien resolvía consultas informativas directas, no podía decidir cómo abordar una tarea compleja, personalizar la respuesta según el estudiante ni ejecutar acciones, limitaciones que motivaron su evolución hacia un agente.

Ventajas del agente frente al modelo RAG

El flujo lineal del RAG resulta insuficiente cuando la consulta exige razonar, combinar fuentes, actuar y recordar. El agente incorpora estas capacidades, tal como se compara en la Tabla 1.

Dimensión	RAG (EP1)	Agente (EP2)
Flujo	Pregunta → búsqueda → respuesta, en un solo paso y sin decisiones	Planifica antes de actuar y re-planifica ante cambios
Capacidades	Solo lectura	10 herramientas de consulta, razonamiento y escritura
Memoria	Historial de 6 mensajes	Memoria de corto y largo plazo (recuerda al estudiante)
Control de calidad	Sin etapa de validación	Validador que contrasta la respuesta con la evidencia
Acciones	No ejecuta acciones	Guarda planes y crea tickets, con autorización previa del estudiante

En términos prácticos, estas diferencias aportan cuatro ventajas principales:

- **Precisión en fechas:** el cálculo de plazos se delega a herramientas en Python, evitando que el modelo realice aritmética de fechas.
- **Personalización:** las respuestas consideran el perfil del estudiante (carrera, jornada, semestre y sede).
- **Memoria:** el agente conserva el contexto dentro de la conversación (ventana de 10 mensajes con resumen sobre 16, mediante *checkpointer*) y entre conversaciones (hechos con *embeddings* recuperados por búsqueda vectorial filtrada por estudiante, además de planes, tickets y trazas).
- **Fiabilidad y derivación:** una respuesta sin respaldo documental es descartada, y los casos sin información disponible se derivan mediante un ticket a coordinación.

Agente que planifica antes de actuar y re-planifica

Se desarrolló un agente basado en el enfoque *plan-and-execute* con ciclo ReAct: en lugar de responder de inmediato, planifica antes de actuar y re-planifica cuando las condiciones cambian. El flujo se compone de cinco fases: planificar, ejecutar (ReAct), revisar, validar y recordar.

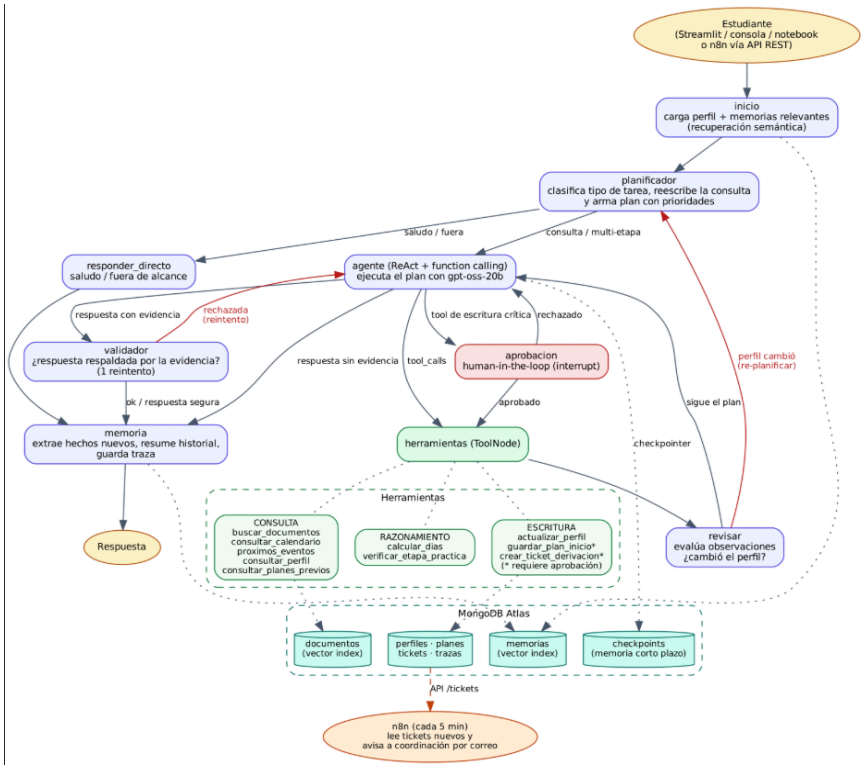
En la fase de planificación, el nodo *planificador* clasifica la consulta (saludo, fuera de alcance, simple, multi-etapa o dato personal), la reescribe con el historial de la conversación y elabora un plan con pasos priorizados. Luego, el nodo *agente* ejecuta el plan mediante *function calling*, decidiendo en cada iteración qué herramienta invocar, y el nodo *revisar* evalúa las observaciones, marca los pasos completados y detecta

si el contexto del estudiante cambió. Si ello ocurre, por ejemplo cuando el estudiante corrige su semestre durante la tarea, el flujo retorna al planificador y se genera un nuevo plan.

Condición	Comportamiento del agente
El estudiante corrige su semestre a mitad de la tarea	Actualiza el perfil y vuelve a planificar
Una herramienta no encuentra resultados	Reformula una vez; si persiste, ofrece crear un ticket
Respuesta sin respaldo en la evidencia	La descarta y reintenta; luego entrega una respuesta segura
Acción de escritura	Se pausa y solicita autorización
Saludo, tema ajeno o intento de cambiar el rol del agente	Ruta corta, sin uso de herramientas
Falla del modelo	Reintenta hasta 3 veces (recursion_limit = 30)

Arquitectura de nodos: un agente, no un sistema multiagente

DuocAIAgent no es un sistema multiagente. La selección de herramientas durante la ejecución del plan recae en un único modelo de lenguaje (gpt-oss-20b), dentro del nodo *agente*. Los demás componentes son nodos de un grafo de LangGraph: etapas con una responsabilidad acotada, conectadas mediante aristas condicionales sobre un estado compartido, cuyo propósito es apoyar al agente para responder las dudas del estudiante. El grafo persiste su información en MongoDB Atlas, y los tickets de derivación son consultados cada 5 minutos por un flujo de n8n que avisa a coordinación por correo electrónico.



La Tabla detalla la función de cada nodo.

Nodo	Función
inicio	Carga el perfil del estudiante y las memorias similares a la consulta desde MongoDB.
planificador	Clasifica la consulta (saludo, fuera de alcance, simple, multi-etapa o dato personal), reescribe la pregunta y arma un plan con pasos.
responder_directo	Responde saludos y temas fuera de alcance sin utilizar herramientas.
agente	El LLM (gpt-oss-20b) ejecuta el plan y decide qué herramienta llamar (ReAct).
aprobacion	Antes de guardar un plan o crear un ticket, detiene el flujo y solicita al estudiante autorizar o rechazar la acción (*human-in-the-loop*).
herramientas	Ejecuta las 10 herramientas disponibles: de consulta, de razonamiento y de escritura.
revisar	Marca los pasos del plan como realizados; si el perfil cambió (por ejemplo, se corrige el semestre), vuelve al planificador para re-planificar.
validador	Verifica que la respuesta esté respaldada por los documentos; si no lo está, la rechaza y el agente reintenta una vez.
memoria	Guarda los datos nuevos del estudiante, resume la conversación si es extensa y registra la traza.

Herramientas utilizadas

La selección tecnológica respondió a los requerimientos de orquestación, persistencia, búsqueda semántica e interacción con el usuario.

Herramienta	Rol en el sistema	Justificación
LangChain	Integración con el LLM y definición de herramientas	Estandariza la invocación del modelo y la definición de herramientas con *function calling*, incluyendo validación de parámetros, lo que reduce el código de integración y facilita el mantenimiento.
LangGraph	Orquestación del flujo del agente	Modela el agente como un grafo con estado compartido, aristas condicionales y ciclos, estructuras necesarias para ReAct y la re-planificación. Incorpora *checkpointer* e interrupciones para el esquema *human-in-the-loop*. Ofrece un control explícito y trazable del flujo.
MongoDB (Atlas)	Persistencia y búsqueda vectorial	Concentra en un solo servicio los documentos y las memorias con índice vectorial, los perfiles, planes, tickets y trazas, y los *checkpoints* de conversación, sin requerir motores separados para datos operacionales y búsqueda semántica.
Streamlit	Interfaz del estudiante	Permite construir una interfaz conversacional web de forma ágil en Python y exponer el razonamiento del agente en un panel desplegable, otorgando transparencia sobre sus decisiones; supera a la consola en usabilidad.
MiniLM multilingüe (local)	Generación de *embeddings*	Soporta textos en español, se ejecuta localmente sin costo por consulta ni dependencia de una API externa y no consume la cuota de tokens del proveedor del LLM.

Como complemento, el razonamiento del agente utiliza el modelo gpt-oss-20b a través de Groq (con gpt-oss-120b como juez en la evaluación), las pruebas automatizadas se implementaron con pytest y la notificación de tickets a coordinación se resolvió con n8n.

Fase de pruebas

La validación del agente se estructuró en tres niveles complementarios:

- **Pruebas automatizadas:** 26 pruebas con pytest que verifican las rutas del grafo con un LLM simulado, permitiendo comprobar la lógica de orquestación sin consumir tokens.
- **Evaluación con *LLM-as-judge*:** 18 casos y 8 escenarios evaluados con gpt-oss-120b como juez, ejecutados mediante *python -m eval.evaluar*, que registra sus resultados en la carpeta *eval/resultados*.
- **Pruebas funcionales de extremo a extremo:** el caso de una estudiante (Camila) recorre cinco escenarios: (1) pregunta simple, como la fecha de toma de ramos del 2026-2; (2) plan multi-etapa, con solicitud de autorización antes de guardarlo; (3) condición cambiante ("voy en 4° semestre"), que activa la re-planificación; (4) consulta sin información disponible, que activa el validador y la derivación mediante ticket a coordinación; y (5) continuidad, en una conversación nueva donde el agente recuerda el plan.

Se definieron cuatro métricas: *planner accuracy* (clasificación y planificación correctas de la consulta), *tool accuracy* (selección de las herramientas esperadas), *answer accuracy* (corrección de la respuesta según el juez) y *retrieval hit@5* (presencia del documento relevante entre los cinco primeros resultados recuperados).

Resultados

La Tabla 5 presenta los indicadores cuantitativos de la evaluación, obtenidos de los archivos generados en *eval/resultados*.

Métrica	Qué mide	Resultado
Planner accuracy	Clasificación y plan correctos de la consulta	96%
Tool accuracy	Herramientas correctas invocadas	83%
Answer accuracy	Respuestas correctas según el juez (gpt-oss-120b)	84%
Retrieval hit@5	Documento relevante entre los 5 primeros resultados	92%

Principal error identificado y ajuste realizado: [Completar tras ejecutar python -m eval.evaluar].

En el caso de uso de demostración, el agente exhibe el siguiente comportamiento: responde consultas simples con respaldo documental; ante una solicitud multi-etapa elabora un plan y solicita autorización antes de guardarlo (archivos .md y .ics); re-planifica cuando el estudiante corrige su semestre; ante ausencia de información impide una respuesta sin respaldo y deriva el caso mediante un ticket a coordinación; y recupera información entre conversaciones, por ejemplo haber reprobado Matemáticas en 2026-1, sin que el estudiante deba repetirla.

Como limitaciones, las etapas de práctica son referenciales, el calendario cubre únicamente el año 2026 y el plan gratuito de Groq impone un límite de tokens.

Conclusiones

El informe deja en evidencia cómo hemos aplicado los conocimientos adquiridos sobre los modelos de inteligencia, modelos RAG, orquestación de agentes de IA. Hemos logrado desarrollar un agente de IA estableciendo la arquitectura, los nodos y tools los cuales puede utilizar el modelo.
Todo este desarrollo nos ha entregado bastante valor, el cual nos aporta como estudiantes y futuros ingenieros al aplicar tecnologías de vanguardia.